

Received 31 December 2024, accepted 23 February 2025, date of publication 27 February 2025, date of current version 6 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3546301

RESEARCH ARTICLE

Scalable 5G NR Rate-Matcher and Rate-Dematcher for Efficient Use in FPGA Accelerators

NEMANJA FILIPOVIĆ^{1,2}, DRAGOMIR EL MEZENI¹, (Member, IEEE),
AND VLADIMIR L. PETROVIĆ^{1,2}, (Member, IEEE)

¹School of Electrical Engineering, University of Belgrade, 11120 Belgrade, Serbia

²Tannera LLC, Santa Monica, CA 90401, USA

Corresponding author: Vladimir L. Petrović (petrovicv@etf.bg.ac.rs)

The work of Dragomir El Mezeni and Vladimir L. Petrović was supported by the Science Fund of the Republic of Serbia (Hybrid Integrated Satellite and Terrestrial Access Network—hi-STAR) under Grant 7750284.

ABSTRACT 5G new radio supports a variety of new technologies and services, demanding significant improvements in radio access network (RAN) latency, throughput, and flexibility. Disaggregation addresses these challenges by splitting the RAN into network units—central (CU), distributing (DU), and radio (RU), enabling data processing virtualization and implementation on off-the-shelf hardware. However, implementing upper physical layer (PHY) processing on off-the-shelf hardware alone might cause inefficient usage of the server processors. Therefore, acceleration is often needed to offload heavy processing, focusing on low-density parity-check (LDPC) codec as the most compute-intensive task in the PHY. Additionally, LDPC coding is tightly coupled with rate-matching. This paper presents novel hardware architectures of rate-matcher and rate-dematcher, targeting field programmable gate array (FPGA) RAN accelerators. The presented solution's approach to memory organization allows highly parallel operation with efficient hardware resource usage. The architecture is flexible, enabling a selection of various parallelism levels for instant integration with other PHY components, and achieves a throughput of up to 150 Gbps for rate-matching, and up to 35 Gbps for rate-dematching. Both components have been integrated into a Peripheral Component Interconnect Express (PCIe) FPGA acceleration card with an LDPC encoder and decoder. The accelerator performance has been evaluated against OpenAirInterface PHY software. By measuring the acceleration impact on the processor load, it has been shown that with the proposed components, the acceleration efficiency can be increased by an order of magnitude compared to the LDPC-only solution.

INDEX TERMS 5G new radio, acceleration, distributed unit, FPGA, high throughput, rate-matching, rate-dematching, radio access network, virtualized RAN.

I. INTRODUCTION

Fifth generation (5G) new radio (NR) [1] is the latest generation of mobile network targeting enhanced mobile broadband (eMBB) along with novel use cases, such as massive machine-type communication (mMTC) and ultra-reliable low-latency communication (uRLLC) [2], [3]. The new standard imposes strict requirements in terms of latency

(e.g., 0.5 ms for uRLLC and 4 ms for eMBB [2]) and throughput (e.g., 10 Gbps of uplink and 20 Gbps of downlink for eMBB [4]). Deployments supporting such features require high processing power hardware and systems with reliable processing latency. These requirements led to the development of disaggregated RAN with functionalities split between multiple units of lower complexity, which also facilitates resource pooling. With standardized interfaces between units, it is made possible to create a RAN deployment with interoperable components from different vendors [5].

The associate editor coordinating the review of this manuscript and approving it for publication was Tiago Cruz¹.

Further evolution of this concept is cloud RAN (C-RAN), where the RAN components are virtualized in the cloud [6], [7]. A 5G split RAN deployment is generally divided into a central unit (CU), distributed unit (DU), and radio unit (RU) [6], [8], [9], [10]. With the possibility of virtualizing both the CU and the DU, current RAN deployments can fully leverage the cloud infrastructure. Since the processing tasks for multiple cells can be aggregated, high-performance accelerators can further increase the energy efficiency and density of such deployments [9].

Several software solutions implement DUs on standard x86 CPUs, such as OpenAirInterface [11], srsRAN [12], or FlexRAN [13]. To increase the energy efficiency and throughput performance of DU implementations, computationally intensive tasks can be offloaded to GPUs [14], [15], FPGA accelerators [9], [16], [17], and specialized processors [18]. Since the low-density parity check (LDPC) codec accounts for most of the computational load in the physical layer processing chain, accelerators offloading the codec offer the highest gain in speed and energy efficiency [9], [19]. Therefore, most of the research on the topic of 5G functionality acceleration focuses on efficient LDPC codec implementations in various technologies [9], [14], [20], [21]. As stated in [9] and [19], the next candidates to offer significant offloading gain are the interleaving and bit selection, and their inverse counterparts, deinterleaving and inverse bit selection. Jointly, interleaving and bit selection form the rate-matching procedure, as defined in [22], while the inverse bit selection and deinterleaving form the rate-dematching procedure. The work in this paper focuses on presenting novel rate-matcher and rate-dematcher architectures that specifically target FPGA or ASIC acceleration of DU functionalities.

Current implementations of rate-matchers and rate-dematchers [23], [24] feature low parallelism levels and limited scalability. A low parallelism architecture can lead to a small hardware footprint but poses specific challenges in high-performance applications like DU accelerators. Specifically, low-parallelism blocks must be instantiated multiple times in parallel to achieve high throughputs. This would fundamentally increase processing latency, buffering, and logic overhead. It will be shown that blocks with higher processing parallelisms are a more efficient solution, both in terms of resource utilization and in terms of processing latency. Furthermore, the data chain parallelism flexibility can enable optimization of hardware usage efficiency for the specific use case, maximizing the processing throughput for the given constraint of the available FPGA resources, e.g., on cloud infrastructure such as [25].

Previous work that is directly comparable to the architecture presented in this paper includes [23], which covers implementations with processing parallelisms of 6-24 bits per transfer. In [24], an implementation with a fixed parallelism of 4 bits per transfer is presented. Both papers focus on bit address generation, which selects individual bits in memory. Achieving higher parallelism levels with bit addressing

increases the required number of memory access ports, logic complexity, and memory primitive utilization. Such implementations show high logic and memory resource usage for the achieved throughput and a considerably smaller range of supported parallelism levels. This makes them a suboptimal choice for both high-performance DU accelerators and lower throughput edge or embedded applications.

Our previous conference work from [26] introduced the basic concepts of the novel memory word addressing scheme, using the example of the rate-dematcher architecture. This paper further expands this addressing scheme to the downlink side, i.e., rate-matcher and presents a detailed architecture and implementation analysis. The proposed hardware architectures offer a high degree of flexibility in terms of throughput and resource utilization. The rate-matcher and rate-dematcher components have been integrated into a forward error correction (FEC) accelerator together with our LDPC encoder [27] and LDPC decoder [21] soft IP cores. Such an accelerator has been evaluated by comparing its performance with software implementations. Additionally, our previous conference work used the naming convention from [23] or [9], where rate-(de)matching and (de)interleaving represent separate functions. This paper aligns the nomenclature with the standard [22], where the rate-(de)matching represents the concatenated (inverse) bit selection and (de)interleaving operations.

The main contributions presented in this paper are as follows:

- 1) definition of novel rate-matcher and rate-dematcher architectures optimized for DU accelerators featuring low latency, scalable throughput, and high hardware usage efficiency (HUE);
- 2) flexibility of the developed components to provide various parallelism levels at compile time and hence enabling an optimal selection for the target use case; This flexibility allows the use both in low-throughput embedded applications and in high-throughput acceleration scenarios
- 3) the integration of developed components into the 5G NR FEC accelerator;
- 4) by an order of magnitude increased acceleration efficiency when compared to the currently presented accelerators in the scientific literature.

The rest of the paper is organized as follows: 5G NR transport block procedures relevant to the presented architectures are described in section II. The theoretical background of the novel addressing scheme that enables efficient implementations of the rate-matcher and rate-dematcher is provided in section III. Hardware architecture and implementation details of the components are presented in IV. Results and discussion are presented in section V. Hardware usage efficiency, maximum operating frequency, and achievable processing throughput are evaluated for multiple parallelisms. An analysis of performance gains in acceleration scenarios is provided. In the end, the conclusion is given in section VI.

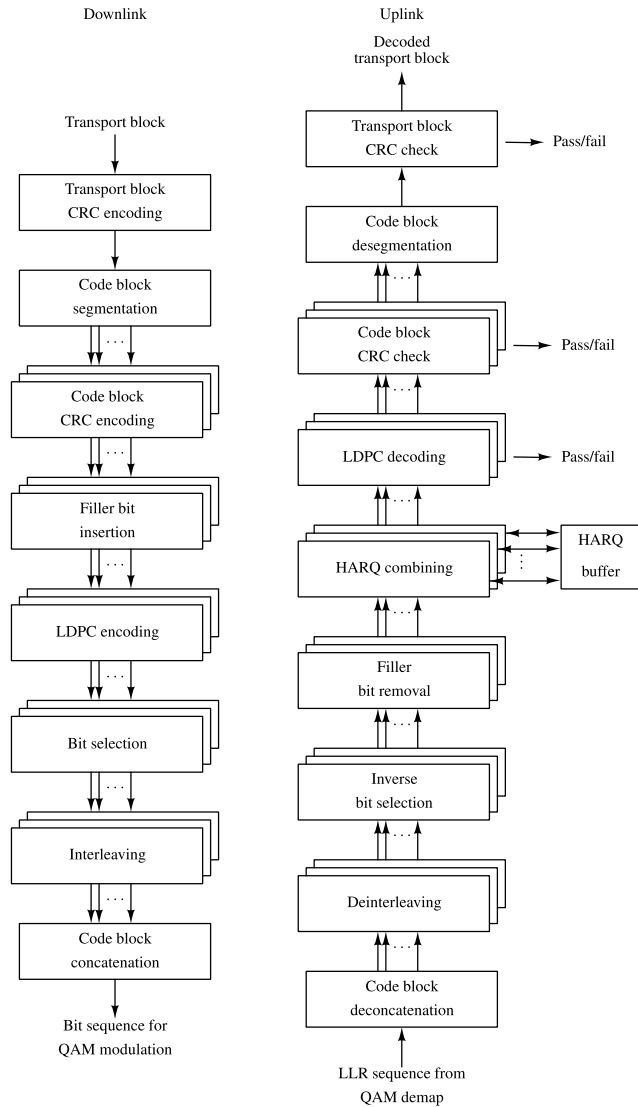


FIGURE 1. Transport block processing procedures.

II. 5G NR TRANSPORT BLOCK PROCEDURES

User data in 5G NR is transferred over a physical downlink shared channel (PDSCH) and a physical uplink shared channel (PUSCH), with data being divided into data units called transport blocks [22]. Transport block (TB) procedures for shared channels at the RAN side are presented graphically in Fig. 1 [22].

At downlink, a cyclic redundancy check (CRC) checksum is calculated and attached to the entire transport block. The stream of bits is then segmented into code blocks for LDPC encoding. A code block CRC checksum is calculated and attached to each information code block. Before LDPC encoding, filler bits are added to match the code block information sequence length with the available LDPC code. After the LDPC encoding, a systematic puncturing is performed. The puncturing is the removal of a number of first bits in the codeword, which correspond to high-weight

columns in the LDPC code's parity check matrix (PCM). The encoded codeword is placed in a circular buffer which is used as the bit selection input. The bit selection step can systematically remove or repeat codeword bits, which matches the transport block size to the available resources allocated for transmission. Some encoded bits are repeated if the target code rate is very low. As defined in the standard [22], the rate-matching also includes bit interleaving. Bit interleaving step increases the SNR performance of the LDPC decoder by spreading the neighboring bits into separate quadrature amplitude modulation (QAM) symbols, thus reducing the effect of burst errors. Values from the bit interleaver are concatenated into a transport block to obtain a bit stream, which is later mapped, modulated, and transmitted.

On the uplink side, an almost symmetrical processing chain is used. The main difference is that the procedures are mainly performed on logarithm likelihood ratio (LLR) values instead of bits. The LLRs are received from the QAM demapper and represent the soft decisions whether the received bit is either "0" or "1". The LLR magnitude indicates the reliability with which the bit was received, whereas its sign represents the assumed bit value. LDPC implementations utilize these properties to achieve significantly better decoding performance when compared to hard decision decoding methods [28]. In the NR processing chain, the LLR stream is split (deconcatenated) into code blocks, which are then passed to the rate-dematching. The rate-dematching performs deinterleaving and the inverse bit selection, including the the LLR combining in case of low code rate repetition. Row-column deinterleaving of LLRs produces the proper LLR sequence. If the bit selection on the transmit (user equipment - UE) side repeated some bits from the circular buffer, the corresponding LLRs on the receiver (g NodeB - gNB) are soft-combined in the inverse bit selection procedure. The obtained LLR stream is passed to the LDPC decoder, which outputs a stream of bits corresponding to the transport block segments. A CRC check of each code block and the entire transport block is performed.

A hybrid automatic repeat request (HARQ) retransmission is requested if an error is detected during the CRC check. A different set of bits is read out from the bit selection buffer [29] in the retransmission. In addition to the soft combining due to the low code rate, the receiver must support the soft-combining of the new set of LLRs with the previously received values to increase the effective SNR of the received sequence. The receiver must store LLRs received in previous transmissions in a HARQ buffer memory. Implemented components fully support this scheme.

The following subsections describe all rate-(de)matching-related TB procedures in more detail.

A. BIT SELECTION OVERVIEW

The upper layer selects the target code rate for LDPC encoding and the modulation order based on the estimated state of the channel. After LDPC encoding, the code word

is written to the circular buffer. Bit selection enables fine adjustment of the code rate and direct control over the transmission sequence length [30]. The redundancy version index (RV_{id}) obtained from the medium access control (MAC) layer dictates which bits are selected for transmission. Each RV_{id} index corresponds to a read start location in the circular buffer [22] and is unique for each HARQ (re)transmission.

LDPC codes in 5G are quasi-cyclic (QC). Their parity check matrix (PCM) $\mathbf{H}$ consists of circularly shifted identity submatrices (circulants) and zero submatrices. The parity check matrices are derived from one of the two base graph matrices (BG1 or BG2) by lifting them to a QC LDPC code PCM. The size of a circulant is called lift factor Z . Therefore, sequence lengths related to the 5G NR LDPC codec are calculated as multiples of Z . The read start location from the circular buffer during bit selection is also a multiple of Z . The read start location $k_{0_{mod}}$ is selected based on the RV_{id} index and the used BG. The $k_{0_{mod}}$ values are given in Table 1.

TABLE 1. Read start location $k_{0_{mod}}$ on BG and RV_{id} selection.

RV_{id}	$k_{0_{mod}}$	
	BG1	BG2
0	$0 \cdot Z$	$0 \cdot Z$
1	$17 \cdot Z$	$13 \cdot Z$
2	$33 \cdot Z$	$25 \cdot Z$
3	$56 \cdot Z$	$43 \cdot Z$

Another mechanism that's employed for a slight increase in coded performance is puncturing. Puncturing is a scheme that removes the first $2 \cdot Z$ bits from the encoded codeword [22]. Additionally, as the size of the transport block is variable, there is a possibility that the code block segment being encoded does not fill the full systematic part of the codeword. In that case, special “null” values are padded and processed by the LDPC encoder as “0”-valued bits. These “null” values are skipped either in the circular buffer read or are removed before writing to the circular buffer. An example of the code block structure obtained from the LDPC encoder component is shown in Fig. 2 [22].

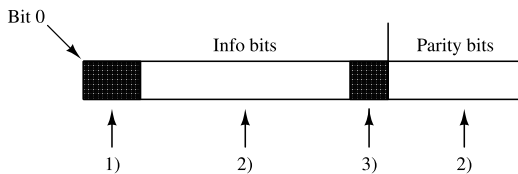


FIGURE 2. Bit-stream content after LDPC encoding: 1) bits for puncturing, 2) bits which are read from the circular buffer, 3) filler/null bits.

The LDPC encoder [27] and the LDPC decoder [21] by the authors of this manuscript were used in conjunction with the components presented in this paper, and they already perform “null” value handling, as well as codeword (de)puncturing. This means that the circular buffer of implemented components contains contiguous bit values

without the need for additional logic to process special locations. If the use of SD-FEC from AMD/Xilinx [31] is desired, an external component for (de)puncturing and filler bit insertion/removal is needed. This component would be beneficial if, for example, the Telco acceleration card from AMD/Xilinx [16] is used as the acceleration platform. The authors have implemented this simple component, but the implementation details are not significant for the scope of this paper.

B. INVERSE BIT SELECTION OVERVIEW

In 5G NR, the inverse of the bit selection operation is performed such that the stream of LLRs from the deinter-leaver is reordered and recombined for LDPC decoding. This operation is symmetrical to the bit selection operation but processes LLR values instead of bits. LLRs have higher resource usage requirements, as they are stored in multiple bits. Typically, LLR values are stored in fixed-point format with bit widths ranging from 6 to 8 [21], [31].

Due to the nature of the bit selection mechanism on the UE side, there is a possibility that some bits have been transmitted multiple times. Corresponding LLRs on the gNB side must then be recombined into a single LLR to be sent to the decoder. This paper uses the bit-level Chase combining (BLCC) algorithm [32] for recombining. The component architecture is designed to be flexible, such that other soft-combining algorithms can be used as long as their processing pipeline is up to several clock cycles long. The BLCC algorithm calculates the LLR value by summing all the received demapper outputs corresponding to retransmitted bits. In fixed-point implementations, this summation must be performed with a saturation step to preserve the LLR sign and the appropriate dynamic range that is expected by the LDPC decoder.

Practically, the inverse bit selection procedure is executed by writing the LLRs starting from location $k_{0_{mod}}$ to the circular buffer. If a particular buffer location is written to multiple times, the LLR values are soft-combined by summation. To simplify the logic, it is possible to initialize the entire circular buffer memory to 0 and always perform saturated additions. This value also facilitates the LDPC decoding process, as the parts of the sequence that are not received should be initialized to 0. LLRs equal to 0 represent a special case where both bit values are equally likely. The high-level overview of inverse bit selection is shown in Fig. 3.

C. HARQ OVERVIEW

A retransmission mechanism is specified in 5G NR [33] to increase the overall link stability. A retransmission occurs if data reception is not acknowledged in time or the receiver sends a “not-acknowledge”. The physical layer then retransmits the data with the next RV_{id} value selected in a hybrid automatic repeat request (HARQ) procedure. The receiver can then soft-combine multiple RV_{id} transmissions that cover different regions of the circular buffer. This

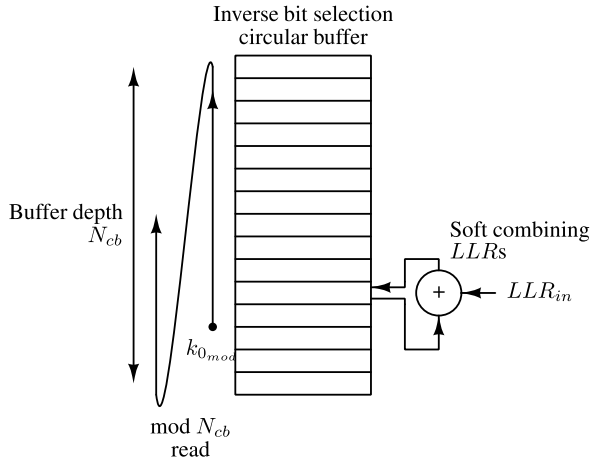


FIGURE 3. Inverse bit selection procedure.

results in more non-zero LLR values becoming available for the decoding and in an equivalent SNR increase for the retransmitted LLR values. Since many other codeword rate-dematch operations can occur between two consecutive retransmissions, the circular buffer contents should be stored in a larger HARQ buffer. The HARQ buffer size is implementation-specific. For FPGA implementations, this buffer is typically implemented using external DRAM, such as DDR4 or HBM [16], [17], [34].

D. INTERLEAVING OVERVIEW

An interleaving block changes the bit order in the LDPC-encoded codeword without changing its content. This operation is useful in spreading the effect of burst errors, which would degrade error correction performance otherwise. In 5G NR, the block interleaving scheme is standardized for use in the data channel. Specifically, a row-column interleaving scheme spreads consecutive bits onto different QAM symbols. As shown in [35], the interleaving improves the error correction performance.

Depending on channel conditions, a modulation order Q_m is chosen from a set of values {1, 2, 4, 6, 8} [36]. Starting from NR release 17 [37], a $Q_m = 10$ was also introduced. Interleaving is performed by writing values from the bit selection circular buffer to the interleaver memory in row-major order. Readout from the interleaver buffer is performed in column-major order, separating the neighboring codeword bits. The number of rows in the interleaver depends on the modulation order and is equal to Q_m . The number of bits read from the buffer E is always an integer multiple of Q_m , so all the interleaver rows are filled. The number of columns corresponds to the number of QAM symbols that are being transmitted and is equal to $\frac{E}{Q_m}$. For BPSK ($Q_m = 1$), there is only one interleaver memory row, so the interleaving operation does not perform any data shuffling.

When the interleaving buffer read/write parallelism is only one bit, the implementation is straightforward and corresponds to the former description. However, with higher

parallelism, which is essential for high throughput operation, bits from multiple locations must be accessed simultaneously to form the output packet. The addressing scheme must be carefully considered to avoid high logic utilization in that case.

E. DEINTERLEAVING OVERVIEW

A deinterleaver in 5G NR is similar in structure to an interleaving component. It inverses the interleaving operation by shuffling LLR values received from the QAM demapper block. Values from the deinterleaver are transferred into the inverse bit selection buffer, which reorders and recombines the sequence into a valid decoder input.

Data that is received from the demapper block is written in column-major order into the deinterleaver buffer. The output is obtained by reading data in a row-major order. The deinterleaver buffer is organized as a matrix $Q_m \times \frac{E}{Q_m}$, in a similar way as the interleaver buffer.

III. EFFICIENT ON-THE-FLY RATE-MATCHING AND RATE-DEMATCHING ARCHITECTURES

This section represents the theoretical foundation of the efficient hardware implementations of rate-matcher and rate-dematcher blocks. To gain buffering and logic utilization efficiency, the functionality of the consecutive components is merged into a single monolithic block. This enables an addressing and shuffling scheme that operates on a single large buffer. The proposed novel approach also allows a processing throughput that does not depend on the selected modulation order Q_m . The suggested method enables the design of scalable components that can process bits/LLRs in parallel with configurable parallelism levels. The parallelism level of the components will be denoted with W . This value specifies the number of bits/LLRs processed in a single transfer. It determines the maximum achievable throughput of the component, total resource usage, and resource usage efficiency.

A. EFFICIENT RATE-MATCHING ARCHITECTURE

The proposed algorithm for efficient rate-matching facilitates simultaneous interleaving and bit selection by following a specific memory access pattern. This pattern is derived by observing the transformation that is performed by concatenated interleaving and bit selection blocks, which are shown in Fig. 4.

In the bit selector, E bits starting from index $k_{0_{mod}}$ are read from the circular buffer. It is assumed that the bit selection circular buffer contains only bits available for transmission. This means the codeword is punctured, and “null” bits are removed.

On the interleaver side, the interleaver memory is configured such that it has Q_m rows and $\frac{E}{Q_m}$ columns. The bit at the start of each interleaver column can be directly addressed in the bit selection circular buffer. By simple evaluation of the operation of both components, this address, denoted as R_i [0]

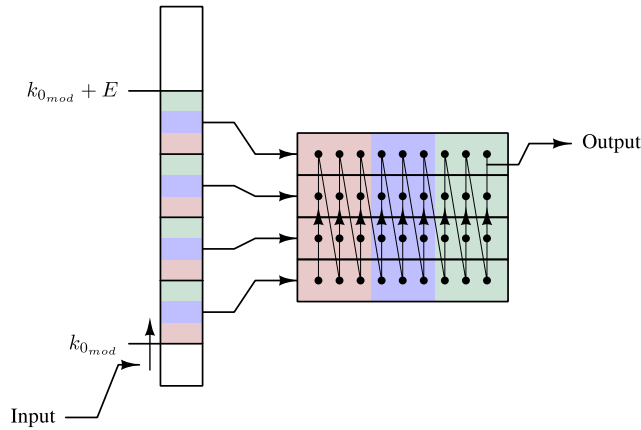


FIGURE 4. Rate-matching scheme used in 5G NR, for $Q_m = 4$. $k_{0_mod} + E < N_{cb}$, and $E = 3WQ_m$.

for row i can be calculated in the following manner

$$R_i[0] = \left(k_{0_mod} + i \cdot \frac{E}{Q_m} \right) \bmod N_{cb}. \quad (1)$$

The address of n -th element in row i can then be obtained in the following manner

$$R_i[n] = (R_i[0] + n) \bmod N_{cb}. \quad (2)$$

Thus, the bits in the first interleaver column bits have addresses $R_0[0]$, $R_1[0]$, $\dots$, $R_{Q_m-1}[0]$ in the circular buffer memory. The next interleaver column can be read by incrementing all addresses from the previous column by 1. If this increment produces an address equal to N_{cb} , a wrap-around to address “0” is performed.

Direct hardware implementation of such an algorithm would require bit-by-bit reads. This can be implemented but would yield a design that utilizes built-in primitives very inefficiently. A similar approach is described in detail in [23]. In that paper, memory is read with the parallelism level of 8 or 6, depending on the selected modulation order.

This paper proposes reads of multiple bits in a single memory access cycle to obtain higher memory access efficiency. This is possible as columns are read out sequentially. Multiple bits from a single row can be read from memory in a single cycle. With Q_m such reads, it is possible to form a block of columns outside of memory iteratively. For example, the blue columns in the interleaver from Fig. 4 can be built up by reading Q_m blue sections from the circular buffer. The size of those sections directly corresponds to the component parallelism. This technique enables flexible access parallelism and maximizes the memory primitive bandwidth utilization while lowering the overall logic complexity.

Practically, high efficiency is achieved by observing the fact that the read operations in (1) and (2) are essentially Q_m concurrent read processes. Each read process accesses values from an interleaver row. The read process can read multiple bits in a single cycle by reading a wider memory word. Bits that are adjacent in an interleaver row are also adjacent in the

bit selection buffer, so they are naturally stored in one or two memory words. The memory parallelism can be matched to the parallelism of the whole component, denoted as W . With a Round-Robin selection of row-read processes, it is possible to sequentially fill a separate, smaller block of interleaver memory with the dimensions of $W \times Q_m$. This interleaver section has the same contents as the full interleaver at the position of the W bit wide window. This corresponds to a section of one color in the interleaver from Fig. 4. Output is built up by reading from that window in column-major order, following the same order as in the standard interleaver. The process repeats with the next window. As the window write and read rates are matched, the next window is available simultaneously as the current window is emptied. Physically, the window contents are stored in flip-flops, and the shuffling is executed by wiring the appropriate window locations to the output interface. Windows are stored in a double buffer. The window buffer will be called the shift out buffer, and the main memory that contains the codeword corresponds to the circular buffer. A top-level architecture leveraging this local shuffling approach is shown in Fig. 5

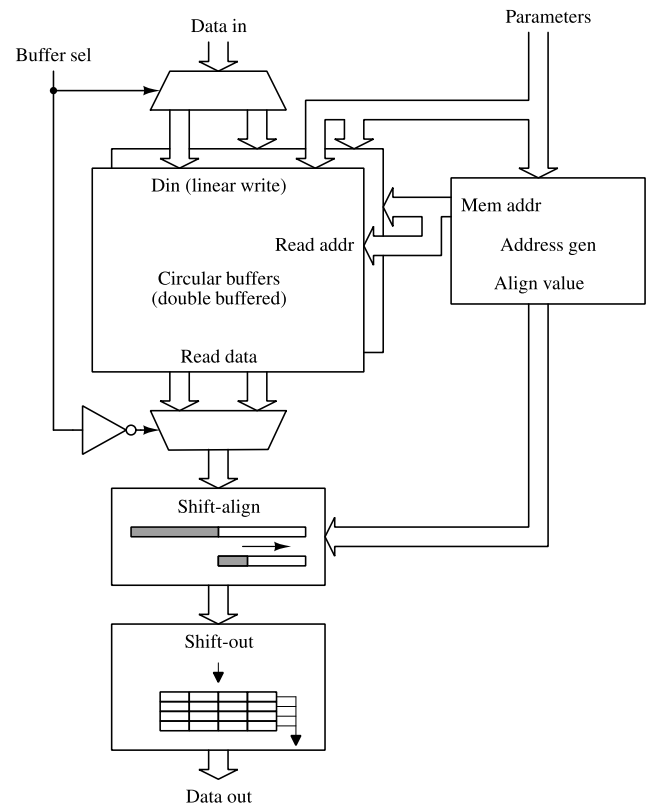


FIGURE 5. Logical organization of the rate-matcher.

In the architecture from Fig. 5, the data is initially written to the circular buffers. As the row section of width W can be stored on one or two memory addresses, the shift-align component isolates the bits that form the row section. Finally, the shift-out buffer performs wired local shufflings such that the rate-matched bit stream is available on the output.

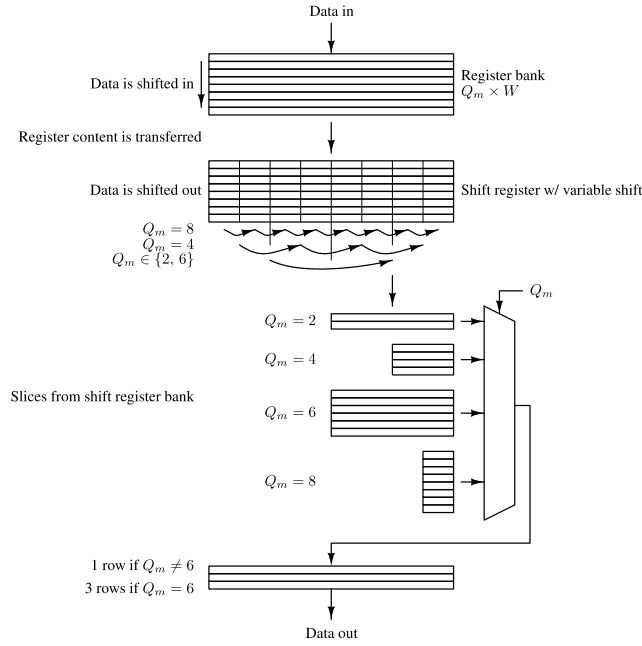


FIGURE 6. Functional diagram of the shift out buffer.

The local wired shuffling, performed in the shift out buffer, is shown in Fig. 6. It enables a significant increase in hardware usage efficiency, which is the main contribution of this paper. The shift out buffer has two register banks that operate in a double-buffered manner. The row sections from the shift-align buffer are written to the input register. Once the register is filled with row segments, its contents are transferred to the output register. Based on the Q_m , an appropriate shift width and output wiring pattern are selected. This enables the forming of Q_m output words. In the case of $Q_m = 6$, 3 output words are formed in local shuffling, as Q_m is not a multiple of W and would otherwise require more complex shifters in the output register matrix. For other Q_m values, the output is formed word-by-word. Sections of the output register matrix used to build up the output word and shift sizes are marked in Fig. 6.

To verify that the read and write rates to the shift out buffer are matched, the duration of both should be evaluated. The shift out buffer stores Q_m words of width W from the circular buffer. Write to the shift out buffer takes Q_m cycles to complete. The buffer read also takes Q_m wired local shufflings to output Q_m words of width W . When a double buffering scheme is used, the described pipeline can operate at the maximum instantaneous throughput of $W \cdot F_{memclk}$. If a simple dual-port memory primitive is used for the circular buffer, the available memory throughput is fully utilized, thus yielding high efficiency. Memory primitive granularity could reduce the hardware usage efficiency, so careful considerations regarding the choice of W are needed to optimize the hardware usage.

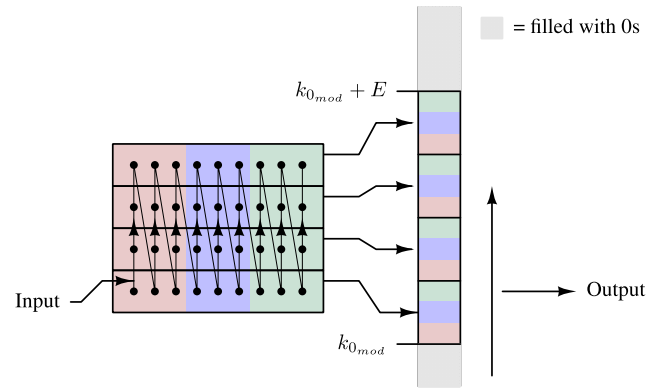


FIGURE 7. Concatenated operation of the deinterleaver and inverse bit selector, for $Q_m = 4$ and $k_{0mod} + E < N_{cb}$.

B. EFFICIENT RATE-DEMATCHING ARCHITECTURE

Rate-dematching is executed in the opposite order to rate-matching. Instead of generating read pointers and performing an operation by reading from memory, the write operations are non-linear, whereas the read is linear. Following the description from the standard [22], the deinterleaver memory has Q_m rows and $\frac{E}{Q_m}$ columns. This memory is filled column-by-column and is read row-by-row to the inverse bit selection circular buffer. Similar to the efficient rate-matching analysis, the mapping of write addresses to the circular buffer memory can be determined by observing the operation of the concatenated deinterleaver and inverse bit selector, shown in Fig. 7.

The addressing pattern is similar to the one described for the efficient rate-matcher operation. The separation between each LLR from a QAM symbol equals the row length $\frac{E}{Q_m}$. Successive QAM symbols are written to incrementing addresses, with preserved wrap-around behavior of the circular buffer. Write address for i -th LLR value of the n -th received QAM symbol can be calculated in the following manner

$$W_i[n] = \left(k_0 + \frac{E}{Q_m} \cdot i + n \right) \bmod N_{cb}. \quad (3)$$

This is symmetrical to the expressions from (1) and (2). Similarly to the rate-matcher, LLR-by-LLR writes with some parallelism level would require a highly granular memory access scheme, lowering the logic utilization efficiency.

A preprocessing step symmetrical to the local shuffling in the interleaver is used to reduce the required number of operations for memory writes. A top-level architecture with efficient memory access is shown in Fig. 8. Partial row segments are built up out of memory in the “shift in” component. The contents of partial row segments are then sent to the shift align buffer sequentially. The shift align buffer aligns the row segments for the memory write, as segments aren’t necessarily word-aligned. This forms larger blocks of consecutive LLR values, which are then written using the full memory width. Such a scheme vastly improves the memory access efficiency.

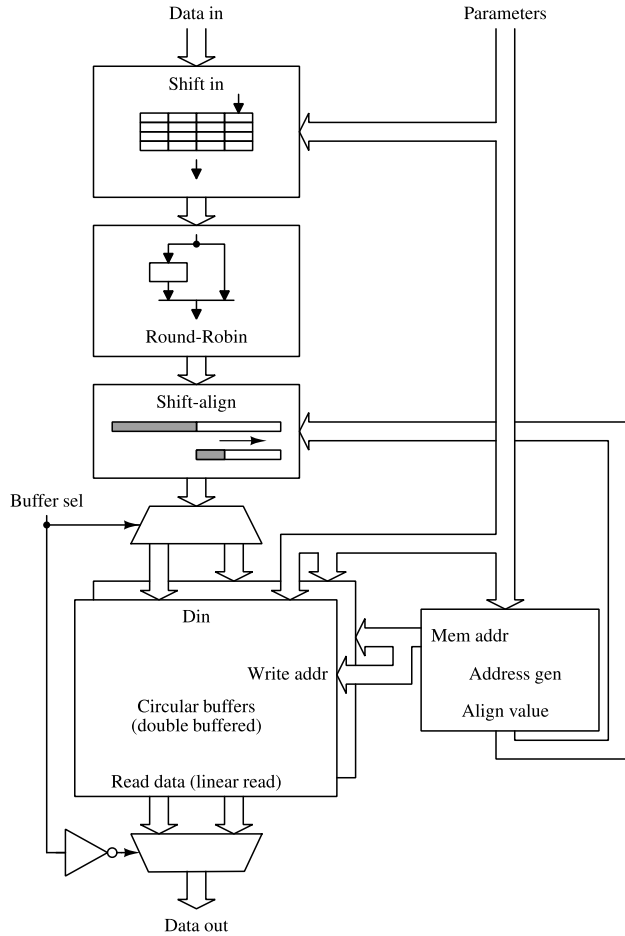


FIGURE 8. Logical organization of the rate-dematcher.

A more detailed view of the shift in component is given in Fig. 9. For modulation orders $Q_m \neq 6$, the input word is split into multiple rows, with row width $\frac{W}{Q_m}$. The row segment register contents are shifted to the left by $\frac{W}{Q_m}$, and the new input value is written to the rightmost section. For $Q_m = 6$, a similar operation is performed, but three input words are used for forming a $\frac{W}{2}$ -wide section. Otherwise, a more complicated shifting control would need to be implemented, decreasing the hardware usage efficiency. Note that this is symmetrical to the operation shown in Fig. 6, but with the difference of operating on LLRs.

Another difference from the rate-matcher implementation is that repeated writes to the same location require LLR soft-combining procedure. For the BLCC scheme (see II-B), the LLRs written to the memory are summed with the existing memory contents. This soft-combining requires simultaneous read and write operations on the same memory address, so a dual-port memory is required for the implementation.

IV. HARDWARE IMPLEMENTATION

A. RATE-MATCHER

The hardware implementation of the rate-matcher leverages the efficient memory addressing scheme, described in III-A. In this section, more details on the concrete algorithm used for

Algorithm 1 Rate-Matching Algorithm

```

1: function Pointer_init( $Q_m, N_{cb}, \frac{E}{Q_m}, k_{0mod}$ )
2:   for  $i \leftarrow 0$  to  $Q_m - 1$  do
3:      $PTR[i] \leftarrow (k_{0mod} + i \frac{E}{Q_m}) \bmod N_{cb}$ 
4:   end for
5:   return  $PTR[0 \dots Q_m - 1]$ 
6: end function
7: function Pointer_increment( $PTR[0 \dots Q_m - 1]$ )
8:   for  $i \leftarrow 0$  to  $Q_m - 1$  do
9:      $PTR[i] \leftarrow (PTR[i] + W) \bmod N_{cb}$   $\triangleright$  Iteratively
10:  end for
11: end function
12: RAM buffer  $MEM[0 \dots \frac{N_{cbmax}}{W}] \times W$   $\triangleright$  Bits
13:  $MEM_{depth} \leftarrow \lceil \frac{N_{cb}}{W} \rceil$ 
14: for  $i \leftarrow 0$  to  $MEM_{depth} - 1$  do
15:    $tmp \leftarrow \text{Get\_current\_input}$ 
16:   if  $i = 0$  then
17:      $First \leftarrow tmp$ 
18:      $MEM[i] \leftarrow tmp$ 
19:   else if  $i = MEM_{depth} - 1$  then  $\triangleright$  Wrap logic
20:      $Shift \leftarrow N_{cb} \bmod W$ 
21:      $MEM[i] \leftarrow tmp \mid (First \ll Shift)$ 
22:      $MEM[i + 1] \leftarrow First \gg Shift$ 
23:   else
24:      $MEM[i] \leftarrow tmp$ 
25:   end if
26: end for
27:  $RP \leftarrow \text{Pointer\_init}(Q_m, N_{cb}, \frac{E}{Q_m}, k_{0mod})$ 
28: Allocate:  $RB[0 \dots Q_m - 1]$   $\triangleright$  Shift out buffer
29:  $shift\_cnt \leftarrow Q_m$  if not  $Q_m = 6$  else2
30: for  $i \leftarrow 0$  to  $\lceil \frac{E}{WQ_m} \rceil - 1$  do
31:   for  $j \leftarrow 0$  to  $Q_m - 1$  do  $\triangleright$  Shift out buffer fill
32:      $SA \leftarrow (MEM[\lceil \frac{RP[j]}{W} \rceil] + 1, MEM[\lceil \frac{RP[j]}{W} \rceil])$ 
33:      $RB[j] \leftarrow SA \gg (RP[j] \bmod W)$ 
34:   end for
35:   for  $j \leftarrow 0$  to  $shift\_cnt - 1$  do  $\triangleright$  Shuffle read
36:     for  $k \leftarrow 0$  to  $\frac{W}{shift\_cnt} - 1$  do
37:       for  $l \leftarrow 0$  to  $Q_m - 1$  do
38:          $out[Q_m \cdot k + l] \leftarrow RB[l][k]$ 
39:       end for
40:     end for
41:     for  $k \leftarrow 0$  to  $\frac{Q_m}{shift\_cnt} - 1$  do
42:        $Write\_to\_output(out[W - 1 \dots 0])$ 
43:        $out \leftarrow out \gg W$ 
44:     end for
45:      $RB \leftarrow RB \gg \frac{W}{shift\_cnt}$ 
46:   end for
47:    $Pointer\_increment(RP[0 \dots Q_m - 1])$ 
48: end for

```

calculating the addresses and performing the shuffling will be given. Function of individual blocks from Fig. 5 will be clarified. An algorithm describing the efficient rate-matching

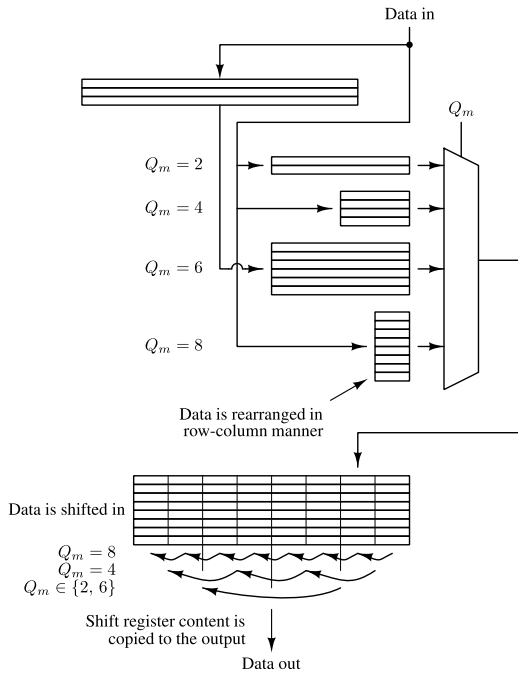


FIGURE 9. Functional diagram of the shift-in buffer of the rate-dematcher.

hardware implementation is given in Algorithm 1. Some of the hardware considerations are abstracted away for clarity. For example, writes and reads are performed in parallel, utilizing the double buffering scheme. All of the sequential operations are pipelined to achieve maximum performance.

Hardware implementation of the component for rate-matching is based on the architecture described in III-A. It facilitates operation in the accelerator environment, with features including code block by code block reconfiguration, low latency, and high memory utilization efficiency. Flexible throughput and utilization tuning are achieved by configuring the parallelism of the component W . An AXI4-Stream top-level interface is used for data, and a separate port with a bit field handles configuration parameters. The configuration is evaluated on the first stream beat of the input codeword.

The design is subdivided into functional blocks, following the functional diagram from Fig. 5:

- Address generator - used for generating the read pointer values
- Circular buffer - handles data storage and write pointer generation. Two of them are instantiated to handle double buffering
- Shift aligner - aligns the bits to the exact pointer location for writing to the output value generating buffer
- Shift out buffer - reshuffles groups of $Q_m \times W$ based on the Q_m in a wired manner such that they are output in the proper order

Top-level control handles handshaking between components and input parameter synchronization. A custom parameter interface that captures parameters on the first AXI4-Stream stream beat is used. The logic is internally

designed to handle the separate control AXI4-Stream if required by the rest of the design. Double buffering is handled by input and output switchover between two identical circular buffers, with a switch occurring on word transition.

1) ADDRESS GENERATOR

The address generator handles memory read pointer generation, as described in equations (1), (2), and Algorithm 1. Initial pointer values are calculated based on input parameters Q_m , $\frac{E}{Q_m}$, and N_{cb} . Operator “ $\text{mod } N_{cb}$ ” is implemented by iteratively subtracting N_{cb} from the result. The increment step for all pointers involves the addition of W to all pointer values and calculating “ mod ”. The initialization step takes at least Q_m clock cycles, with additional cycles used for $\text{mod } N_{cb}$ operations. Pointer increment usually takes one clock cycle, accounting for zero or one $\text{mod } N_{cb}$ operation per increment. If a rarely used word configuration in which $W > 2 \cdot N_{cb}$, the pointer increment takes longer. To avoid implementing the division and modulo logic for arbitrary divisor values, a constraint on $W = 2^n$ is introduced, as it vastly simplifies div and mod by W .

2) CIRCULAR BUFFER

The circular buffer handles the storage of the encoded codeword for interleaving. Its functionality handles the operations with the MEM structure from Algorithm 1. The circular buffer stores the parts of the codeword marked for transmission, with puncturing and null bit removal already handled. As the data arrives from the encoder component via the AXI4-Stream interface, with a parallelism of W , it is written to consecutive memory addresses. Upon the last transfer, the component performs write finalization and then transitions to read mode. The write finalization extends the last memory write by appending the shifted values from the first transfer to the last valid bit of the last transfer. This enables simple handling of the pointer wrapping when reaching the top of the buffer. This process handles lines 19-22 from Algorithm 1. Externally passed parameters generate the control values required for proper circular repetition and state transitions.

For reads from the circular buffer, addresses are fed from the address generator. The component iterates through the generated row pointers sequentially. The rest of the components down the data processing pipeline (shift-align and shift-out) operate assuming this Round-Robin-like row read order. Since the top-level component works in a continuous streaming mode, the circular buffer is instantiated twice (with one operating in write and the other in read mode). Multiplexing and demultiplexing of control and data streams are handled on the top level.

3) SHIFT ALIGNER

This component facilitates parallel fractional addressing required by the read pointers. The $\log_2 W$ least significant bits of the address select the first row segment bit in the selected memory word. A pipelined shift register aligns this bit with

the row segment start. To ensure that W bits are available after shifting, the circular buffer reads two consecutive memory locations. This provides $2W$ bits for shifting, satisfying the bit ordering (see line 32 in Algorithm 1). Concatenated bits from the circular buffer are fed into a pipelined shifter, with the shift value being directly connected to the lower bits of the address. The output of this component is a slice of lower W bits of the shifter output (see line 33). Since the full output width is not required, the shift stage size in the pipeline can be reduced, to further lower the resource usage.

4) SHIFT OUT BUFFER

The shift-out buffer performs the local row-column interleaving of row segments. The row segments are shifted into an intermediate buffer containing all of the data for the output shuffling. After Q_m rows are received, they are transferred to a shift register with a programmable shift size (shift size $\in \{\frac{W}{2}, \frac{W}{4}, \frac{W}{8}\}$). Note that the number of shifts is always an integer since W is a power of 2. Reordering is then performed by wiring the lower part of the register to an output register that holds values in linear order. For $Q_m \in \{2, 4, 8\}$, the output can be wired directly, since W is a multiple of Q_m . In that case, the shift can be performed completely without stitching or introducing more possible shift sizes into the shift register configuration. This does not include the $Q_m = 6$ configuration, as it does not divide W .

For $Q_m = 6$, register shift size is set to $\frac{W}{2}$, and the right part of the shift out register, with the dimensions of $6 \times \frac{W}{2}$ is used for forming the output in linear fashion. The output has $3 \cdot W$ bits and is shifted out over 3 output AXI4-Stream beats. Since the input procedure takes 6 stream beats, and the shift out can be executed in 2 cycles, with each cycle forming 3 output stream beats, the maximum achievable throughput of $W \frac{\text{bits}}{T_{\text{clk}}}$ is maintained. A similar method for $Q_m = 10$ can be used to update the component for the newest releases. This would require an output register holding 5 output stream beats. This implements lines 35-46 from Algorithm 1.

A constraint of $W \geq 8$ is introduced to keep the component implementation simple. Otherwise, the proposed addressing scheme would require more buffering and logic for building row segments. In the FPGA, the shift register with variable shift size is implemented such that it shares the flip-flop resources and the shift size is selected by a multiplexer on the FF input. The output register has a similar multiplexer configuration, with the control logic handling the number of rows in it.

B. RATE-DEMATCHER

The hardware implementation of the component for rate-dematching is based on the scheme described in III-B. It facilitates operation in an accelerator environment with runtime word reconfiguration and flexible parallelism levels. In our previous work [26], we briefly described the basic concepts of the addressing scheme for the rate-dematcher.

Algorithm 2 Rate-Dematching Algorithm

See lines 1-11 from Algorithm 1 for Pointer_init and Pointer_increment implementations. W corresponds to deinterleaver width. All elements, and shifts are performed on the unit of 1 LLR. This is not written explicitly in the algorithm, for clarity.

- 1: **RAM buffer** $MEM \left[0 \dots \frac{N_{cbmax}}{W} \right] \times W$ $\triangleright$ LLRs
- 2: $MEM_{depth} \leftarrow \left\lceil \frac{N_{cb}}{W} \right\rceil$
- 3: $WP \leftarrow \text{Pointer_init}(Q_m, N_{cb}, \frac{E}{Q_m}, k_{0mod})$
- 4: **Allocate:** $WB[0 \dots Q_m - 1], WB_{1D}[0 \dots Q_m - 1]$
- 5: $shift_cnt \leftarrow Q_m$ **if** not $Q_m = 6$ **else** 2
- 6: $TR[W - 1 \dots 0] \leftarrow 0$
- 7: **for** $i \leftarrow 0$ **to** $\left\lceil \frac{E}{WQ_m} \right\rceil - 1$ **do**
- 8: **for** $j \leftarrow 0$ **to** $shift_cnt - 1$ **do**
- 9: **for** $k \leftarrow 0$ **to** $\frac{Q_m}{shift_cnt}$ **do**
- 10: $tmp \leftarrow \text{Get_current_input}$
- 11: $in \leftarrow in \ll W$
- 12: $in[W - 1 \dots 0] \leftarrow tmp$
- 13: **end for**
- 14: $WB \leftarrow WB \ll \frac{W}{shift_cnt}$
- 15: **for** $k \leftarrow 0$ **to** $\frac{W}{shift_cnt} - 1$ **do**
- 16: **for** $l \leftarrow 0$ **to** $Q_m - 1$ **do**
- 17: $WB[l][k] \leftarrow in[Q_m \cdot k + l]$
- 18: **end for**
- 19: **end for**
- 20: **end for**
- 21: **for** $j \leftarrow 0$ **to** $Q_m - 1$ **do**
- 22: $SA \leftarrow (WB_{1D}[j], WB[j])$
- 23: $tmp \leftarrow SA \gg (WP[j] \bmod W)$
- 24: $ptr \leftarrow \left\lfloor \frac{WP[j]}{W} \right\rfloor$
- 25: $MEM[ptr] \leftarrow MEM[ptr] + tmp[W - 1 \dots 0]$
- 26: **if** $(WP[j] \geq N_{cb} - W)$ **then**
- 27: $TR \leftarrow TR + tmp[2W - 1 \dots W]$
- 28: **end if**
- 29: **end for**
- 30: $WB_{1D} \leftarrow WB$
- 31: Pointer_increment($WP[0 \dots Q_m - 1]$)
- 32: **end for**
- 33: $MEM[MEM_{depth} - 1] \leftarrow MEM[MEM_{depth} - 1] + TR$
- 34: **for** $i \leftarrow 0$ **to** $MEM_{depth} - 1$ **do**
- 35: Write_to_output($MEM[i]$)
- 36: $MEM[i] \leftarrow 0$
- 37: **end for**

This paper will further clarify the addressing scheme and present the concept in more depth.

An algorithm describing the efficient rate-dematching implementation is shown in Algorithm 2. Most hardware-related specifics, such as pipelining and double buffering, are abstracted for clarity. The top-level structure of the rate-dematcher is similar to the rate-matcher component but with a reversed order. Data is rearranged for non-linear writes to the circular buffer, and reading from the circular buffer

is performed linearly. Furthermore, the component processes soft values (LLRs) in fixed-point representation. This means data processing equivalents from the interleaver side are replicated over the LLR bit-width.

Regarding the interfaces, the component data streams are implemented following the AXI4-Stream protocol, whereas the word configuration is sent via a separate port, with the configuration packet registering on the first stream beat of the input data stream.

Top-level design is subdivided into the following functional blocks:

- 1) Address generator - generates the write pointer addresses, as well as aligner shift values. Implemented in the same way as the address generator for the rate-matcher.
- 2) Shift in buffer - performs local reshuffling to form row segments for writing to memory
- 3) Round-Robin selector - iterates through the shift in buffer and registers the previous buffer values
- 4) Shift align - performs fractional addressing of the LLRs by shifting the values, forming the aligned memory word
- 5) Write-add circular buffer - handles data storage and soft combining of LLR values. It is instantiated twice for continuous operation

1) ADDRESS GENERATOR

This component is implemented identically as the equivalent component on the interleaver side. There are only differences in circular buffer dimension constants that do not affect the implementation. For more information, see subsection IV-A1.

2) SHIFT-IN BUFFER

This component performs local row-column deinterleaving of LLRs into row segments that are used for writing to memory. It performs an inverse operation to the shift-out buffer component from the rate-matcher. Input stream beats are sliced into Q_m rows, with the hard-wired row-column deinterleaving pattern, forming a $Q_m \times W$ matrix of LLRs. Each row holds LLRs adjacent in the circular buffer memory space. The hard-wired input fills the lowest part of the shift-in buffer after each input stream beat. That content is shifted to the left by $\frac{W}{2}$, $\frac{W}{4}$, or $\frac{W}{8}$ LLRs, depending on the value of Q_m . After 2, 4, or 8 shift operations (occurring with the input stream beats), the buffer is full, and its contents are copied to the round-robin selector.

Similarly to the interleaver, the special case exists when $Q_m = 6$, as it is not a divisor of W . A smaller buffer holding 3 input stream beats is used in that case. When it is filled, the buffered content is shuffled similarly as for other configurations, with the shift size of $\frac{W W_{LLR}}{2}$. The functional diagram of the shift-in buffer is shown in Fig. 9. In case of the need for supporting $Q_m = 10$ for newer releases, a similar buffer that stores 5 input stream beats can be used.

3) ROUND-ROBIN SELECTOR

This component forms the row segments of the width $2 W W_{LLR}$ for shifting. The previous copy of the row segments is also buffered here. The rows are sent to the output iteratively after receiving the new packet from the shift-in buffer. Output to the shift aligner takes Q_m clock cycles, which matches the Q_m beats required for the shift-in buffer fill. As the shift aligner requires the current and previous row segment value, this component also handles the row segment buffering.

4) SHIFT ALIGN

This component performs the fractional addressing of rows to form valid circular buffer memory words. The internal pipelined shift register performs shifting with the granularity of W_{LLR} since that is the addressable unit. In contrast to the equivalent component on the rate-matcher side, this component also handles the separate buffering for wrap-around conditions. In the rate-matcher, the write finalization step copies over the first written address and appends it after the last written bit. This enables the wrap-around read to be performed in the same manner as before, with addresses $\lfloor \frac{N_{cb}}{W} \rfloor$, and $\lfloor \frac{N_{cb}}{W} \rfloor + 1$ being read in the same clock cycle. This was feasible in the rate-matcher implementation since the dual-port nature of the BRAM primitive was used to read two consecutive memory locations. In the rate-dematcher implementation, because of the requirement for soft-combining of LLRs, one of the ports is used for the read, whereas the other is used to write the soft-combined value. This means that the memory address $\lfloor \frac{N_{cb}}{W} \rfloor$ would need to be written (and soft-combined) with partially populated LLR values.

To bypass the more complicated control logic that would be needed in that case, as well as to reduce the overhead on wrap-around, a separate register is used for emulating the memory location $\lfloor \frac{N_{cb}}{W} \rfloor$. This value is then soft-combined with the value on memory location 0 when a read is performed. Write to this register is performed on wrap-around condition, and values that are written to it are the higher W bits from the internal shift register. The top level controls writes to that register. This top register handles lines 26-28 from Algorithm 2.

5) WRITE-ADD CIRCULAR BUFFER

This component performs the buffering and soft-combining of received LLR values. It is equivalent to the circular buffer from the rate-matcher but with a reversed role. Data is written to it non-linearly, using the pointers from the address generator, whereas the readout is performed in linear order. LLR values equivalent to multiple transmission bits are added together following the BLCC scheme. It should be noted that this operation does not handle HARQ retransmission, but just the case where the bit selection configuration satisfies $E > N_{cb}$.

For soft-combining, the write operation is performed in multiple steps:

- 1) Memory read from the write address; initializing address delay line
- 2) Soft combining with the set of W LLRs arriving from the shift aligner
- 3) Check for collisions in the address delay line. If there is a read from the same address in progress, tag that pipeline stage.
- 4) Write to the memory with collision check pipeline update

To maximize the operating frequency of the design and reduce the place and route effort, the memory primitives are instantiated with a 2 clock cycle read latency. This somewhat complicates the implementation of the component since there is a possibility of collisions when a relatively small N_{cb} is configured. A collision is a condition where the memory read has occurred, and a new request is issued for the same memory location with the old data still in the processing pipeline. This would cause the outdated memory contents to enter the processing pipeline, and give an invalid inverse bit selection output if not properly handled.

A collision occurs when there is a need to read from the memory address multiple times in a period shorter than 4 clock cycles for this implementation. In that case, the read pipeline is populated with an outdated value. Because of this, two separate delay lines follow the data flow. The first is populated with the set of memory addresses from which reads occurred. A read collision is identified if another read is issued from a memory address contained in the delay line. In that case, an appropriate stage in the collision identification pipeline is set to “1”. When the data with an identified collision arrives at the soft combiner, it is exchanged with the value written to the memory and stored in a separate register.

For sending the data to the output, there is an internal counter for iterating through read memory addresses. When a memory address is read out, it is wiped by writing all “0”. This means that the next word can be soft-combined with the emptied contents unconditionally.

This component is used in a double-buffered configuration, with one performing writes and the other performing reads. The top-level state machine takes care of multiplexing/demultiplexing data streams. Internally, the component has a read and a write state.

V. RESULTS AND DISCUSSION

Most of this section focuses on FPGA synthesis, placement, and routing results obtained in Vivado 2021.1, targeting a Zynq UltraScale+ RFSoc XCZU28DR device. The components were written in VHDL 1993, using the standard Vivado design flow and default synthesis settings. The device is a good equivalent of the XCZU48DR available on the Telco T2 card from AMD, as they share a similar fabric. XCZU28DR was available for authors in the lab for direct

hardware testing. This equivalence was confirmed using the Vivado device database.

All synthesis results were functionally verified against the 3GPP-compliant test software in a hardware-based FPGA testbench utilizing PYNQ environment [38]. The lower bound on parallelism was set to $W = 8$ for both the interleaver and the deinterleaver, which is functionally defined, as specified in previous sections. The upper bound on W was set at the point where “place and route” process fails due to very high complexity. This also sets the upper bound on maximum achievable throughput with the proposed design on Versal UltraScale+ architecture.

Maximum clock frequency was determined following the guide from AMD [39]. The highest achievable clock frequency constraint was found using a binary search. The best non-negative WNS and F_{clk} values from the out-of-context implementation timing report were used in the F_{max} calculation.

Functional verification was performed using a simple DMA-based IP integrator project, with the use of standard Vivado library IP cores for glue logic between the processor and the DUT. The block diagram of the test design is shown in Fig. 10.

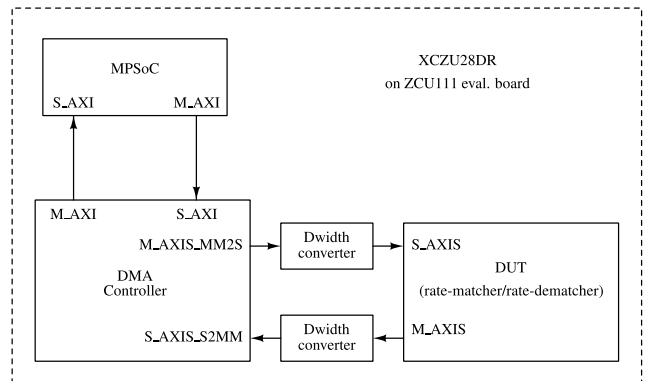


FIGURE 10. Test design used for verification in hardware.

Additionally, the design has been tested in a lookaside FEC accelerator for RAN deployments. A lookaside accelerator is an accelerator configuration that only exchanges data with the accelerated software. In contrast, an inline accelerator communicates with software on one end and with lower RAN components via onboard interfaces on the other end.

For lookaside accelerator evaluation in this paper, a Data-Plane Development Kit (DPDK)/Baseband Device (BBDEV)- compliant FEC accelerator was implemented on an Alveo U50 PCIe FPGA [40] card from AMD/Xilinx with a custom UltraScale+ FPGA onboard. The DPDK library facilitates high-performance driver development for network interface cards (NIC) and other accelerators. Among the supported accelerators is a Baseband Device class that covers lookaside accelerated baseband processing for 3GPP-compliant deployments. The accelerator design offloads FEC for 5G NR on a code-block level. This design was used as a

reference to compare the performance metrics to a readily available OpenAirInterface implementation [41].

A. RATE-MATCHER IMPLEMENTATION AND PERFORMANCE

The rate-matcher component was synthesized for parallelism levels $W \in \{8, 16, 32, 64, 128, 256, 512\}$. The router of the Vivado tool could not complete the routing task due to extremely high routing congestion for $W = 1024$. Regarding the practicality of the synthesized configurations, some systems may require extremely low processing latency in this part of the data chain, necessitating very high parallelisms. In practice, the component was highly optimized to operate with the parallelism level of $W = 128$. This parallelism roughly targets the throughputs of encoders from [27] or [31]. Since the encoder component is usually the throughput bottleneck in practical accelerator implementations [9], a rate-matcher that matches the encoder throughput presents a good tradeoff from hardware utilization perspective. If a very low latency is needed, higher parallelisms could offer faster processing, but with the rate-matcher hardware itself being underutilized.

Hardware utilization and maximum achieved clock frequencies for multiple parallelism levels W are shown in Fig. 11.

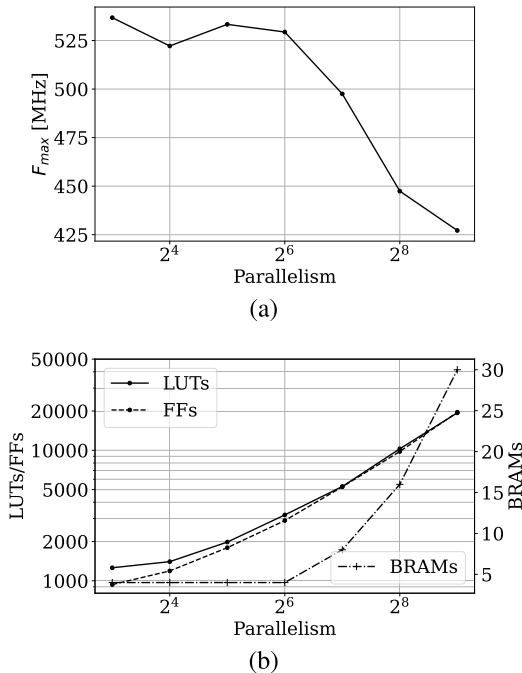


FIGURE 11. Rate-matcher maximum clock frequency (a) and utilization reports (b).

Regarding the maximum operating frequency of the design, for parallelisms 128 and less, the synthesis tool gives results that are close to the theoretical maximum of the UltraScale+ architecture [42].

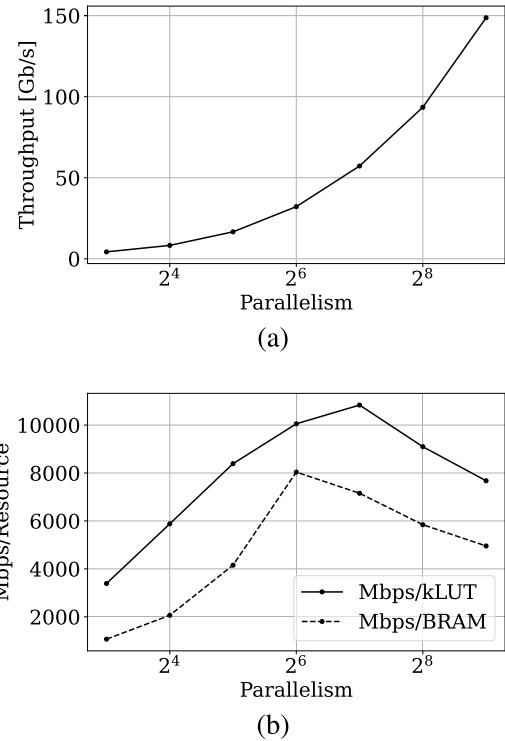


FIGURE 12. Measured throughput (a) and hardware usage efficiency (b) of the rate-matcher various parallelisms.

The steady-state throughput of this implementation should scale directly with the parallelism, as the streaming interfaces can output W bits per clock cycle without stalling during the interleaving state. Practically, this is optimistic, as there is an overhead in calculating the pointer values and other parameters that are needed during operation. This means that the average throughput will vary based on the use-case, with changes in codeword length and E . In this analysis, the throughput measurement was performed with fixed word configuration $N_{cb} = 20000$, $E = Q_m \left[\frac{N_{cb}}{Q_m} \right]$, and $k_{0mod} = 4992$. The overhead in recalculating the parameters depends only on the parameter values, so this analysis with a fixed configuration was deemed adequate in assessing real-world hardware performance. The measured throughput and the obtained hardware usage efficiency are shown in Fig. 12.

Observing the LUT usage efficiency plot of the interleaver, there is a peak at $W = 128$. Further analysis of the implementation results shows that the cause for this at higher W s is the result of the lower maximum frequency and the aggressive primitive replication. The constant logic (e.g. address generator) causes the rise in LUT usage efficiency at lower W s, as its overhead drops with a wider datapath. The BRAM usage efficiency starts dropping with the rise in BRAM count due to the port width requirement. Up to that point, BRAM is underutilized from the throughput standpoint as it is cascaded. As the BRAM count rises, it becomes underutilized from the memory capacity standpoint. BRAM

TABLE 2. Comparison with similar work, for $W = 8$.

Work	LUT	FF	BRAM	F_{max} [MHz]	Parallelisms
This	1259	936	4	536.8	8 - 512
[23]	7364	614	16	400	6 - 24
[24]	30921	32085	16	100	4

primitives are best utilized at $W = 64$ when the entire memory space is utilized, and the entire port width is used.

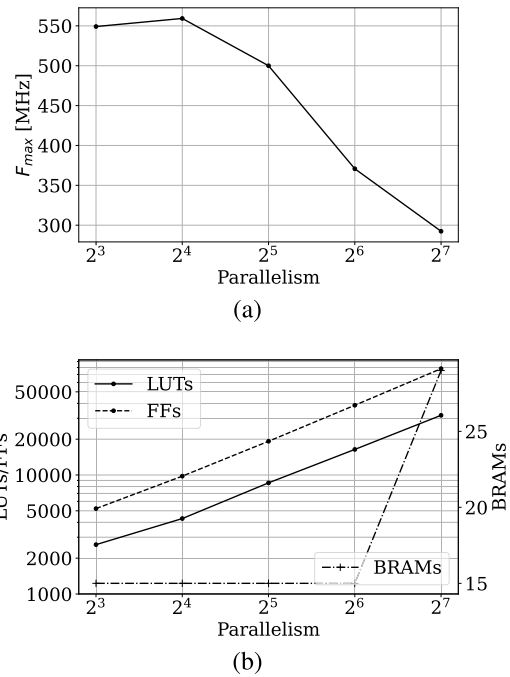
Within the existing body of research, the work presented in [23] represents the closest match to the functionality implemented in this paper. Reference [23] implements the functionality with the memory access parallelism of 8. Work from [23] targets the UltraScale+ architecture, so the primitives are equivalent. Another work featuring a rate-matcher implementation with a different memory access scheme is shown in [24]. In that paper, the circular buffer content is accessed with a parallelism of 4, with multiple circular buffer copies. Reference [24] also implements the filler bit removal component. Even though a different scheme is used, the processing throughput directly corresponds to an equivalent configuration with $W = 4$. A detailed comparison of the implementations is provided in Table 2.

It should be noted that the read logic from [23] does allow some scalability but with a prohibitive increase in routing complexity and a requirement for a higher number of memory replicas. Regarding the scalability of the component from [24], it was shown that the parallelism level of 8 also leads to a prohibitive increase in implementation complexity.

B. RATE-DEMATCHER IMPLEMENTATION AND PERFORMANCE

The rate-dematcher component was synthesized for $W \in \{8, 16, 32, 64, 128, 256\}$, with the synthesis tool failing at $W = 512$. The practical use of parallelism levels is skewed to lower values than the interleaver, as the decoding chains typically feature lower throughputs than the encoding chains. Decoding chains also often operate on LLR values, with the LLRs being presented in fixed-point format. The presented implementation operates with 8-bit wide LLR values, and it can handle both the implementations from [21] and [31]. Other LLR formats are fully supported, including hard decisions (bits), without any changes in functionality.

With the same considerations as for the interleaver in subsection V-A, the operating frequency optimization was performed for $W = 16$. This matches the throughput of the LDPC decoders from both [21], and [31]. Good utilization efficiency of the deinterleaver hardware is achieved by matching the parallelism and throughput as closely as possible to the decoding component. In that case, the deinterleaver will not lower the throughput of the decoder, while the deinterleaver complexity is kept as low as possible. The decoder is usually the most complex component in the processing chain [9], so this deinterleaver sizing strategy is a good optimization of hardware usage efficiency. If a low processing latency is needed, the deinterleaver can be

**FIGURE 13.** Rate-dematcher maximum clock frequency (a) and utilization reports (b).

oversized, lowering the number of clock cycles the deinterleaving operation takes. In that case, the deinterleaver would have longer idle periods, lowering the overall hardware usage efficiency of the design. The system design based on this implementation should take into account the implementation results and optimize the parallelism level to the required use case.

The hardware utilization and maximum achieved clock frequencies for synthesized parallelisms are shown in Fig. 13.

The maximum achieved clock frequency is near the theoretical maximum for parallelisms $W \in \{8, 16\}$. This aligns well with the conclusions drawn for the interleaver. As the deinterleaver is operating on units that are 8 times larger, the amount of raw bits processed per clock cycle is the same for the highest efficiency cases for both components.

The throughput was measured the same way as the throughput for the interleaver. The initial address calculation overhead introduces the same stalls between two words. This overhead matches the overhead for the interleaver, as the same address calculation logic is used. The configuration for which the throughput was measured is $N_{cb} = 20000$, $E = Q_m \left[\frac{N_{cb}}{Q_m} \right]$, and $k_{0mod} = 4992$. The measured throughput and the calculated hardware usage efficiency for synthesized configurations are shown in Fig. 14.

Observing the LUT usage efficiency of the deinterleaver, it peaks at $W = 16$. Similarly to the interleaver, the rise before the peak is caused due to the constant-logic overhead. That constant logic overhead includes the address generator, control handshake interfaces, state machines, and other miscellaneous logic. The fall in hardware usage

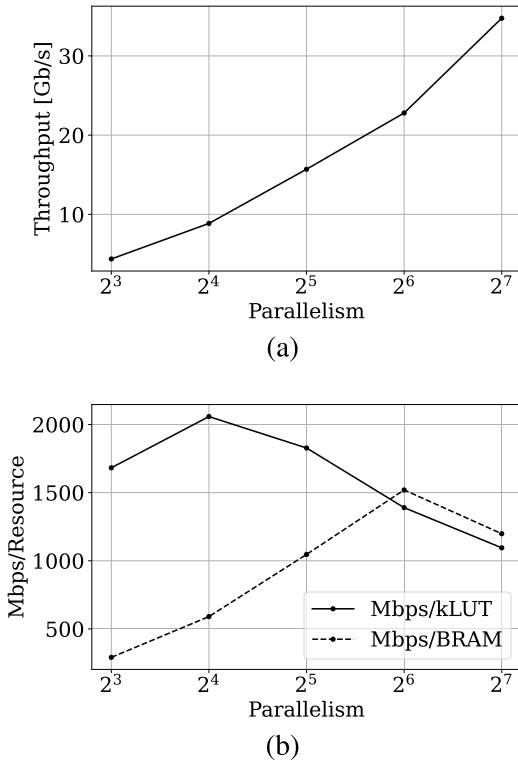


FIGURE 14. Measured throughput (a) and hardware usage efficiency (b) of the rate-dematcher for various parallelisms.

efficiency after the peak is caused by the fall in maximum operating frequency and logic replication introduced by the synthesis tool. Regarding the BRAM hardware usage efficiency, a similar consideration to the interleaver stands. Due to the larger width of the data unit, the peak RAM usage efficiency is at $W = 64$. Before that parallelism, the BRAM blocks are cascaded and underutilized from the throughput standpoint. After that parallelism level, the required BRAM throughput leads to an underutilization from the memory fill standpoint. Since the memory access scheme is the same for both the interleaver and the deinterleaver, the maximum usage efficiency parallelism is the same for both ($W = 64$).

In our previous work [26], only the $W = 16$ configuration was analyzed, with the achieved operating frequency of 400 MHz. Compared to that paper, some clock frequency optimizations were introduced to bring the maximum operating frequency at $W = 16$ up to 559.3 MHz.

C. DDPK/BBDEV-COMPLIANT FEC ACCELERATION

The rate-matcher and rate-dematcher components were integrated into an FPGA-based lookaside accelerator that was integrated into a 3GPP-compliant end-to-end system. The FPGA lookaside accelerator was implemented on an Alveo U50 card PCIe card from AMD/Xilinx. The card is a data center accelerator card that features PCIe Gen4 $\times$ 16 connectivity as the primary interface. For LDPC encode and decode cores, designs described in [21] and [27] were

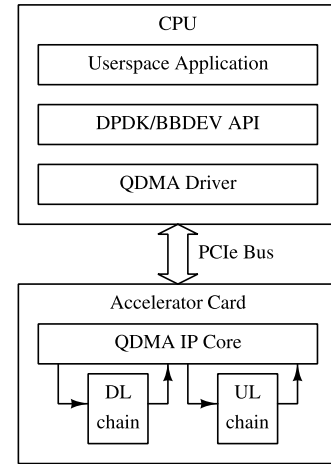


FIGURE 15. Block diagram of a DDPK/BBDEV-compliant lookaside acceleration system.

used. This paper will not describe other code block processing cores, but the measurements are performed on a fully functional design. The Alveo U50 card contains a custom UltraScale+ FPGA device, with the Vivado design flow remaining unchanged. It was confirmed that the out-of-context implementation results on this platform match the results presented in previous subsections.

The hardware design features processing chain elements that perform code-block level encoding and decoding. Interfacing is implemented through a DDPK/BBDEV-compliant driver on the processor side and a QDMA subsystem for PCI Express on the FPGA side. A block diagram of the system is shown in Fig. 15.

To compare the performance against existing solutions, the OpenAirInterface [41] was used as a representative software implementation. For both hardware and software, code-block processing function calls were timed. This enabled the direct comparison of two implementations, as the library API function calls were measured. The measurements accounted for all overheads, including data transfers, code block configuration, and pipeline stalls. For OpenAirInterface measurements, `ulschsim` and `dlschsim` utilities were used. Slight modifications to the performance measurements in both utilities were made to enable the measurement of individual functionalities. The throughput was measured using one thread. It should be noted that the OpenAirInterface enables multithreaded processing, but the assumption that the throughput rises linearly with the number of available threads will be made. Generally, a scheduling overhead could reduce the multithreaded performance slightly, but this analysis is outside this paper's scope. The measurements were executed on an AMD Ryzen 7 3800X Processor operating at 3.9 GHz [43]. Results are shown in Table 3.

Values from Table 3 were measured and calculated by timing the processing steps in both the accelerator and software. For software measurements, an average from $1 \cdot 10^4$ runs of the same configuration were used. RM/RDM

TABLE 3. Results of the software and hardware throughput measurements.

Processing direction	bg	k	R	TBS [bits]	Available bits	RM/RDM share [%]	SW [Mbps]	HW [Gbps]	Equiv. CPUs (LDPC + RM)	Equiv. CPUs (LDPC only)
Encoder	1	8448	1/3	7808	21000	17.7	48.7	9.8	200.4	35.4
				31752	88452	44.8	121.4	9.8	80.5	36.0
			8/9	7936	8964	12.0	53.8	18.3	340.7	40.9
				65576	74880	46.2	204.4	18.3	89.7	41.5
	2	1280	2/3	1192	1800	11.4	37.5	3.0	78.6	9.0
			1/5	1224	6408	23.3	33.0	1.5	46.0	10.7
Decoder	1	8448	1/3	7808	21000	5.2	4.4	1.8	396.8	20.5
				7936	8964	3.6	6.9	4.6	663.1	23.7
			2/3	1192	1800	4.6	6.6	2.5	380.6	17.6
				1224	6408	10.2	4.5	1.0	220.0	22.4
	2	1280	1/5	1224	6408	10.2	4.5	1.0	220.0	22.4
			1/5	1224	6408	10.2	4.5	1.0	220.0	22.4

bg: LDPC code's basegraph; k: number of info bits; R: target code rate; TBS: Transport block size in available info bits; Available bits: Number of coded bits available in the channel grant; RM/RDM share: Total share of the rate-matcher (DL) or the rate-dematcher (UL) in the code block processing time; SW: Measured software information throughput; HW: Measured hardware information throughput; Equiv. CPUs (LDPC+RM): Number of SW threads needed to achieve the same HW throughput; Equiv. CPUs (LDPC only): Number of SW threads needed to achieve the same HW throughput when LDPC processing is accelerated but rate-(de)matching is kept in software

share was calculated by dividing the total processing time by the processing times of the rate-matcher or the rate-dematcher blocks in the transport block processing call. For hardware, an average from $5 \cdot 10^6$ runs was used for throughput measurement. The software and hardware decoders were configured to decode with 6 LDPC iterations. The accelerator performs all calculations in a pipelined manner, and code block reconfiguration was performed for each encoding. Code block configuration is passed to the hardware processing IP as a small header that contains all relevant parameters. The hardware internally strips the header, and a control logic drives configuration inputs of all components in the chain appropriately. This implementation uses the fast streaming interface for both control and data, significantly reducing overhead. All measurements have shown that the processing throughput is mainly governed by the processing throughput of the LDPC block.

The equivalent number of CPUs was calculated by dividing the achieved HW throughput by the achieved SW throughput. For the equivalent number of CPUs with LDPC acceleration, the achieved hardware processing throughput was divided by the processing throughput of the software rate-matcher and rate-dematcher. As the LDPC is the slowest block on the hardware side, it was assumed that the LDPC accelerator would perform roughly the same as the code block processing accelerator.

The design was tested with several representative transport block configurations. The coderate, LDPC codeword size, and the total transport block sizes were selected by configuring the modulation coding scheme (MCS) and channel grant sizes in the `ulschsim` and `dlchsim` calls. MCS defines the target coderate and the grant modulation order. Grant size and the modulation order determine the number of bits available for over-the-air transmission. TBS is the total number of MAC bits processed in the transport block chain.

Channel grants were configured to cover high and low coderates in both BG1 and BG2. Regarding the grant configurations, significant effort was made to provide

realistic examples. To demonstrate smaller LDPC codeword sizes and the use of BG2, only small channel grants with exactly 1 LDPC codeword can be used. Otherwise, larger LDPC codewords and BG1 will be used for transmission [33]. Including transport block configuration procedure from [33] in the evaluation of code block processing performance enables a realistic assessment of performance in a RAN deployment that isn't present in other relevant work on this subject. For example, in [9], performance of LDPC encoding with BG2 with multiple codewords encoded in the same grant was evaluated, increasing the SW performance significantly. This configuration is not possible.

The OpenAirInterface LDPC encoder can process 8 LDPC codewords from a transport block in parallel. If smaller transport blocks are used, the throughput is significantly impacted. This speedup cannot be used for BG2 codewords, as transport blocks have only 1 LDPC codeword. This is evident in processing throughput differences for the first 4 rows of Table 3 (where $k = 8448$). The processing throughput for larger grants was significantly higher.

The rate-matcher processing time with higher software LDPC encoding throughput becomes significant. For example, in one configuration 46.2 % of the transport block processing time was used by rate-matching. Furthermore, this creates a bottleneck in the accelerated DL processing. To feed a single LDPC accelerator, up to 42 CPU threads would need to handle rate matching. By facilitating the offload of the rate-matching step, the achievable downlink accelerator speedup can be increased by approximately 8x in some configurations. A significant increase in processing speedup is evident in all downlink configurations.

In the uplink processing chain, the rate-dematcher represents a less significant part of the data processing time. In contrast to the range of 12 - 46 % on the downlink side, the rate-dematcher takes 3.6 - 10.2 % in the example transport block configurations. These values are due to comparatively low LDPC decoding throughput when compared to the hardware implementation. If the equivalent CPUs metric is

observed, it can be seen that the code block offload represents a speedup of more than 600x when compared to software. This speedup would not be possible without offloading the rate-dematching block, as the software rate-dematcher becomes the bottleneck. This means that the total accelerator speedup is increased by a factor of approximately 20-30 in most tested DL configurations due to rate-dematching offload.

VI. CONCLUSION

The design of the 5G NR-compatible rate-matcher and rate-dematcher for use in FPGA-accelerated DUs was successfully implemented and verified. The scalability of the presented design was demonstrated by implementing all possible configurations. Implementation results that would be relevant for selecting the required parameters are given in great detail. This provides a good selection guide for the potential full integration of this design. Configurations ranging from a 1.3 kLUT and 4.3 Gbps up to 19 kLUT and 150 Gbps are available for use in the PDSCH datapath. For the PUSCH datapath, there is a selection of configurations ranging from 2.6 kLUT and 4.4 Gbps up to 32 kLUT and 35 Gbps. These values represent the highest reported throughputs as well as the widest available configuration ranges in literature.

The implemented components were tested in silicon, using field-proven 3GPP-compliant software for generating test vectors. The interfaces of the design were made with integration considerations, following industry-standard protocols and facilitating easy reconfiguration for real-world use cases.

To evaluate the performance increase due to code block processing offload, the rate-matcher and rate-dematcher were integrated into a standard-compatible FPGA design. It has been shown that offloading only the LDPC processing block to a dedicated accelerator has significantly lower throughput than offloading the entire code block processing. Furthermore, it has been demonstrated that even the low processing share of the rate-dematching on the uplink side becomes a bottleneck in an accelerated deployment. On the downlink side, it has been shown that rate-matching can take almost half of the transport block processing time in some configurations. This analysis highlighted the need for high-performance and scalable rate-matching and rate-dematching processing blocks for hardware acceleration in 5G NR cloud-RAN network deployments.

The work presented in this paper presented a novel architecture of rate-matcher and rate-dematcher required for practical accelerated RAN deployments. It has been successfully demonstrated that significant performance gains can be obtained by offloading more functionalities to a dedicated FPGA or ASIC accelerator block. The presented architecture is designed in such a way that it can be deployed both on high-performance and resource-constrained parts, such as programmable SoCs.

As a future work, further modifications of the architecture to support additional channels in 5G NR, such as in [35], could increase the efficiency of the overall L1 processing acceleration. The introduction of parallelism levels that are not power of 2 could also increase the flexibility and further aid the integration. However, the implementation complexity in this case could also increase, requiring additional tradeoff analysis.

The proposed accelerator could be integrated into other systems or acceleration scenarios. For example, it is interesting to evaluate the acceleration efficiency in low-power resource-constrained deployments such as small cells. On the other hand, the system can be integrated into the inline accelerator for even higher speedups.

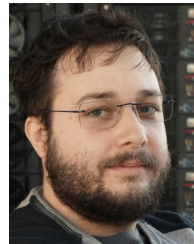
ACKNOWLEDGMENT

The authors would like to express gratitude to Tannera LLC and Miloš Marković, Senior Engineer at Tannera, for various types of support during the project.

REFERENCES

- [1] *Digital Cellular Telecommunications System (Phase 2+) (GSM); Universal Mobile Telecommunications System (UMTS); LTE; 5G; Release Description; Release 15*, document (TR) 21.915, Version 15.0.0, 3GPP, Oct. 2019.
- [2] *Study on Scenarios and Requirements for Next Generation Access Technologies*, document (TR) 38.913, Version 16.0.0, 3GPP, Jul. 2020.
- [3] S. Parkvall, E. Dahlman, A. Furuskar, and M. Frenne, "NR: The new 5G radio access technology," *IEEE Commun. Standards Mag.*, vol. 1, no. 4, pp. 24–30, Dec. 2017.
- [4] *Minimum Requirements Related to Technical Performance for IMT-2020 Radio Interface (S)*, document M.2410-0, M. ITU-R, 2017.
- [5] 5G Americas. (2020). *Transition Towards Open & Interoperable Networks*. [Online]. Available: <https://www.5gamericas.org/wp-content/uploads/2020/11/InDesign-Transition-Toward-Open-Interoperable-Networks-2020.pdf>
- [6] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits proposed for 5G mobile crosshaul networks," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 146–172, 1st Quart., 2019.
- [7] A. Checko, H. L. Christiansen, Y. Yan, L. Scolari, G. Kardaras, M. S. Berger, and L. Dittmann, "Cloud RAN for mobile networks—A technology overview," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 1, pp. 405–426, 1st Quart., 2015.
- [8] *Study on New Radio Access Technology: Radio Access Architecture and Interfaces*, document (TR) 38.801, Version 14.0.0, 3GPP, Mar. 2017.
- [9] E. A. Papatheofanous, D. Reisis, and K. Nikitopoulos, "LDPC hardware acceleration in 5G open radio access network platforms," *IEEE Access*, vol. 9, pp. 152960–152971, 2021.
- [10] Y. Alfidhli, P.-C. Peng, S. Shen, Y. Shuang, M. S. Omar, Y.-W. Chen, S. Mitani, and G.-K. Chang, "Towards dynamic 5G networks utilizing flexible function split," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Aug. 2019, pp. 1–3.
- [11] F. Kaltenberger, A. P. Silva, A. Gosain, L. Wang, and T.-T. Nguyen, "OpenAirInterface: Democratizing innovation in the 5G era," *Comput. Netw.*, vol. 176, Jul. 2020, Art. no. 107284.
- [12] (Nov. 2024). *SrsRAN Project—Open Source RAN*. [Online]. Available: <https://www.srslte.com/>
- [13] X. Foukas, N. Nikaein, M. M. Kassem, M. K. Marina, and K. Kontovasilis, "FlexRAN," in *Proc. 12th Int. Conf. Emerg. Netw. Experiments Technol.*, Dec. 2016, pp. 427–441.
- [14] C. Tarver, M. Tonnemacher, H. Chen, J. Zhang, and J. R. Cavallaro, "GPU-based, LDPC decoding for 5G and beyond," *IEEE Open J. Circuits Syst.*, vol. 2, pp. 278–290, 2021.
- [15] (Nov. 2024). *NVIDIA Aerial*. [Online]. Available: <https://developer.nvidia.com/aerial>
- [16] *Xilinx T2 Telco Accelerator Card*, AMD/Xilinx, San Jose, CA, USA, 2021.

- [17] Tannera 5G RAN FEC Acceleration IP. Accessed: Nov. 2024. [Online]. Available: <https://tannera.io/5g-ran-fec-acceleration-ip>
- [18] NXP Layerscape Access LA12xx Programmable Baseband Processor. Accessed: Nov. 2024. [Online]. Available: <https://www.nxp.com/products/processors-and-microcontrollers/arm-processors/layerscape-processors/layerscape-access-la12xx-programmable-baseband-processor:LA12xx>
- [19] E. A. Papatheofanous, D. Reisis, and K. Nikitopoulos, "The LDPC challenge in software-based 5G new radio physical layer processing," in *Proc. IEEE Int. Medit. Conf. Commun. Netw. (MeditCom)*, Sep. 2021, pp. 312–317.
- [20] C.-H. Lin, C.-X. Wang, and C.-K. Lu, "LDPC decoder design using compensation scheme of group comparison for 5G communication systems," *Electronics*, vol. 10, no. 16, p. 2010, Aug. 2021.
- [21] V. L. Petrovic, M. M. Markovic, D. M. E. Mezeni, L. V. Saranovac, and A. Radošević, "Flexible high throughput QC-LDPC decoder with perfect pipeline conflicts resolution and efficient hardware utilization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 12, pp. 5454–5467, Dec. 2020.
- [22] *Multiplexing and Channel Coding*, document (TS) 38.212, Version 16.2.0, 3GPP, Mar. 2020.
- [23] K. C. Behera, "An efficient low-latency algorithm and implementation for rate-matching and bit-interleaving in 5G NR," in *Proc. IEEE 3rd 5G World Forum (5GWF)*, Sep. 2020, pp. 565–571.
- [24] R. Jain, N. Anjum, S. Soujanya, H. Rudramuniyappa, A. Jain, A. Bisht, E. Sharma, and P. Singh, "FPGA implementation of rate matching in 5G NR PDSCH," in *Proc. IEEE Region 10 Conf. (TENCON)*, Oct. 2023, pp. 1–6.
- [25] Amazon. (2024). *Amazon EC2 F1 Instances*. [Online]. Available: <https://aws.amazon.com/ec2/instance-types/f1/>
- [26] N. Filipovic, D. El Mezeni, and A. Radošević, "Hardware implementation of 5G NR deinterleaver and de-rate matcher," in *Proc. 15th Int. Conf. Adv. Technol., Syst. Services Telecommun. (TELSIKS)*, Oct. 2021, pp. 57–60.
- [27] V. L. Petrović, D. M. E. El Mezeni, and A. Radošević, "Flexible 5G new radio LDPC encoder optimized for high hardware usage efficiency," *Electronics*, vol. 10, no. 9, p. 1106, May 2021.
- [28] R. Jose and A. Pe, "Analysis of hard decision and soft decision decoding algorithms of LDPC codes in AWGN," in *Proc. IEEE Int. Advance Comput. Conf. (IACC)*, Jun. 2015, pp. 430–435.
- [29] *Physical Layer Procedures for Control*, document (TS) 38.213, Version 16.2.0, 3GPP, Jul. 2020.
- [30] F. Hamidi-Sepehr, A. Nimbalkar, and G. Ermolaev, "Analysis of 5G LDPC codes rate-matching design," in *Proc. IEEE 87th Veh. Technol. Conf. (VTC Spring)*, Jun. 2018, pp. 1–5.
- [31] *Soft-Decision FEC Integrated Block LogiCORE IP Product Guide (PG256)*, AMD/Xilinx, San Jose, CA, USA, 2022.
- [32] M. El-Khamy, J. Lee, and I. Kang, "Soft turbo HARQ combining," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2013, pp. 5542–5547.
- [33] *Physical Layer Procedures for Data*, document (TS) 38.214, Version 16.2.0, 3GPP, Jul. 2020.
- [34] Intel. (2024). *Intel Vran Accelerator ACC100*. [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/211796/intel-vran-accelerator-acc100-adaptor/specifications.html>
- [35] X. Xiong, Y. Dai, Z. Hu, K. Huo, Y. Bai, H. Li, and D. Liu, "Hardware sharing for channel interleavers in 5G NR standard," *Secur. Commun. Netw.*, vol. 2021, pp. 1–13, Jan. 2021.
- [36] *Physical Channels and Modulation*, document (TS) 38.211, Version 16.2.0, 3GPP, Jul. 2020.
- [37] *Physical Channels and Modulation*, document (TS) 38.211, Version 17.1.0, 3GPP, Apr. 2022.
- [38] AMD/Xilinx. (2024). *Pynq Python Productivity TY AMD Adaptive Compute Platforms*. [Online]. Available: <https://www.pynq.io/>
- [39] *UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)*, AMD/Xilinx, San Jose, CA, USA, 2024.
- [40] *Alveo U50 Data Center Accelerator Card User Guide (UG1371)*, AMD/Xilinx, San Jose, CA, USA, 2023.
- [41] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, "OpenAirInterface: A flexible platform for 5G research," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 5, pp. 33–38, Oct. 2014.
- [42] *Zynq UltraScale+ MPSoC Data Sheet: DC and AC Switching Characteristics (DS925)*, AMD/Xilinx, San Jose, CA, USA, 2024.
- [43] (Nov. 2024). *AMD Ryzen7 Processors for Desktops*. [Online]. Available: <https://www.amd.com/en/products/processors/desktops/ryzen.html>



NEMANJA FILIPOVIĆ was born in Belgrade, Serbia, in 1996. He received the B.S. and M.S. degrees in electrical and computer engineering from the University of Belgrade, Serbia, in 2019 and 2020, respectively, where he is currently pursuing the Ph.D. degree with the School of Electrical Engineering.

In 2019, he joined Tannera LLC, where he is a Senior Hardware Design Engineer. His research interests include digital signal processing, VLSI designs, and SoC implementations of communication systems.



DRAGOMIR EL MEZENI (Member, IEEE) was born in Belgrade, Serbia, in 1985. He received the B.S. degree in electrical engineering, and the M.S. and Ph.D. degrees in electronics from the University of Belgrade, Serbia, in 2008, 2010, and 2018, respectively.

He is currently an Assistant Professor of electrical engineering with the University of Belgrade. His research interests include digital image/video processing, computational photography, and very large-scale integration architectures for digital signal processing.



VLADIMIR L. PETROVIĆ (Member, IEEE) received the Dipl.-Ing., M.S., and Ph.D. degrees in electrical engineering from the University of Belgrade, Serbia, in 2014, 2015, and 2021, respectively.

He is currently an Assistant Professor of electrical engineering with the School of Electrical Engineering, University of Belgrade, and a CTO with Tannera LLC. His research interests include VLSI design, communication systems architectures, digital signal processing, and hardware implementations of signal processing algorithms.

...